Meta Pre-Training World Action Models for Few-Shot Robot Post-Training

Anonymous CVPR submission

Paper ID *****

Abstract

World action models (WAMs) that jointly generate future video and robot actions are pre-trained on web-scale video and cross-embodiment robot data, and are then post-trained on demonstrations of a target robot and environment. This recipe works well when hundreds of demonstrations are available, but we show that it degrades sharply in the practical regime where only a handful of demonstrations per task can be collected: post-training the mid-trained Cosmos3-Edge model on three demonstrations per task reduces the LIBERO success rate from 87–99% to 49–83%. We argue that mid-training on a multi-embodiment mixture is not the same as learning to adapt *from few demonstrations, and we close this gap with a few-shot meta pre-training stage inserted between mid-training and post-training. Using only the robot datasets already present in mid-training, the stage repeatedly simulates few-shot post-training—k optimizer steps of the exact post-training recipe on K demonstrations of one source embodiment—and moves the weights toward the adapted solution with the first-order Reptile update. Because the update is a plain interpolation in parameter space, all 1.4B trainable parameters of the post-training recipe can be meta-trained on four commodity GPUs with the standard sharded training stack, and the resulting checkpoint is a drop-in replacement for the mid-trained weights: the post-training recipe itself is unchanged. With three demonstrations per task, the meta pre-trained initialization improves the success rate on LIBERO-10, -Goal, -Spatial, and -Object from 58.0/48.8/72.0/82.8% to 69.4/60.8/72.6/85.2%, improving every one of the ten LIBERO-10 tasks. The benefit is not limited to the few-shot regime: with all demonstrations, LIBERO-10 improves from 87.2% to 94.6%.*

1. Introduction

Robot learning is converging on a two-stage recipe. A large model is first pre-trained on web-scale video and on cross-embodiment robot data, and is then *post-trained* on demonstrations collected with the target robot in the target environment. World action models (WAMs), which generate future video and robot actions jointly within one network [7, 9, 36, 46, 53], are the latest instance of this recipe: the video objective provides a dense, embodiment-agnostic training signal, and the action objective turns the same network into a policy. When the post-training set is large, the recipe is remarkably effective. On the four LIBERO suites [29], post-training the mid-trained Cosmos3-Edge WAM [36] on all available demonstrations reaches 87–99% success (Fig. 1, blue).

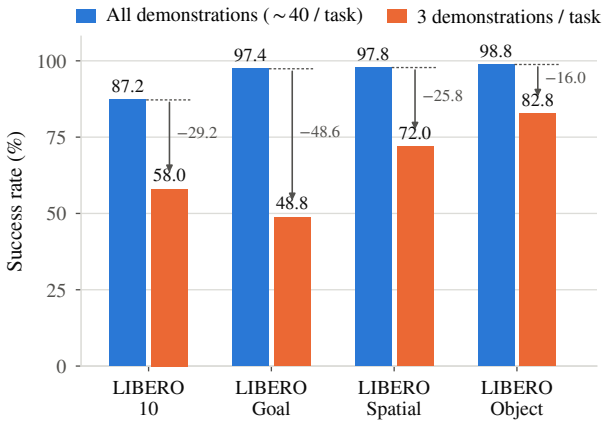


Figure 1. **Few-shot post-training collapses.** Closed-loop success rate of the mid-trained Cosmos3-Edge world action model after post-training on all demonstrations of each LIBERO suite (blue; 379–454 demonstrations, about 40 per task) and on three demonstrations per task (orange; 30 per suite), with the post-training recipe otherwise identical. The same recipe loses 16 to 49 points when demonstrations are scarce. Every bar is 50 trials on each of the 10 tasks of a suite.

Demonstrations, however, are the most expensive ingredient of this pipeline. Every new robot, workcell, or task requires a person to teleoperate it, and in many deployments only a handful of demonstrations per task is affordable. We therefore ask how the standard recipe behaves when the post-training set is small, and define a *few-shot post-training* protocol: three demonstrations per task, *i.e.*, 30 demonstrations per LIBERO suite, with the post-training recipe otherwise unchanged. The answer is sobering. Suc-

cess drops by 16 to 49 points (Fig. 1, orange), to 58.0% on LIBERO-10 and 48.8% on LIBERO-Goal. A world model that has been trained on millions of robot and human videos cannot turn 30 demonstrations into a reliable policy.

Why does so much prior knowledge help so little? Post-training updates 1.4B parameters of the generator tower and the action heads on roughly 7.6k training windows, and the action head of the new embodiment is initialized from scratch. More fundamentally, mid-training optimizes the weights for the *joint* objective over a mixture of embodiments, and nothing in that objective asks the weights to be a good *starting point* for a short fine-tune on an embodiment outside the mixture. Being good at many embodiments and being quick to adapt to a new one are different properties. This distinction is the premise of gradient-based meta-learning [11, 33], which so far has been applied to small policies on a few tasks of one robot rather than to billion-parameter world models.

We bring this idea to WAMs with a few-shot meta pre-training stage inserted between mid-training and post-training (Fig. 2). The stage uses no new data: it draws episodes from five robot datasets that are already part of the mid-training mixture, each recorded with a different embodiment. In every episode it simulates the downstream problem, K demonstrations of one source embodiment and k steps of the exact post-training optimizer, loss, and trainable set, and then moves the weights a fraction of the way toward the adapted solution: the Reptile update [33]. Reptile is well matched to this scale. Its update is first-order and needs neither a query set nor a second forward pass at the adapted point, so the model's own sharded parameters serve as the fast weights, the standard FSDP training stack [50] runs the inner loop unchanged, and the only extra state is one copy of the trainable parameters. The resulting checkpoint θ_{meta} is a drop-in replacement for the mid-trained weights, and we post-train from it with the unchanged recipe.

On the four LIBERO suites with three demonstrations per task, post-training from θ_{meta} instead of the mid-trained weights raises the success rate from 58.0/48.8/72.0/82.8% to 69.4/60.8/72.6/85.2% and improves every one of the ten LIBERO-10 tasks. The initialization also helps when data is plentiful: with all LIBERO-10 demonstrations the success rate rises from 87.2% to 94.6%. In summary, our contributions are:

- We identify and quantify the few-shot post-training problem of WAMs with a three-demonstrations-per-task protocol on the four LIBERO suites, on which the standard recipe loses 16–49 points.
- We propose few-shot meta pre-training, a Reptile-based stage between mid-training and post-training that optimizes the entire trainable set of the post-training recipe for few-shot adaptation, needs no additional data, and leaves the architecture and the post-training recipe unchanged.
- We show consistent gains on all four suites in the few-shot regime (+6.6 points on average, up to +12.0) and a +7.4 point gain with full data.

2. Related Work

2.1. Few-Shot Learning

Meta-learning. Few-shot learning asks a model to solve a new task from a few examples, and meta-learning makes this ability the training objective [16]. Metric-based methods compare queries to a support set in a learned embedding space [42, 43]. Optimization-based methods instead learn an initialization from which a few gradient steps suffice. MAML [11] differentiates through the inner loop, its first-order variant drops the second-order terms, and Reptile [33] replaces the meta-gradient by an interpolation toward the weights reached after k inner steps, which decouples the inner horizon from the memory footprint. ANIL [40] shows that much of MAML's benefit on few-shot classification comes from feature reuse with a rapidly adapting head, and [1] documents the instability of MAML-style training. We use Reptile, but with an inner loop that is the full post-training recipe of a 1.4B-parameter trainable set rather than a few steps on a small network, and we rely on the fact that its update is a convex combination of weights, which lets the standard sharded training stack run the inner loop.

Few-shot imitation and data-efficient fine-tuning. In robotics, one-shot imitation learning conditions a policy on a demonstration [10, 19], and meta-imitation learning applies MAML to visuomotor policies so that a single demonstration adapts them [12, 49]; skill-based methods adapt a pre-trained skill prior to a few demonstrations [15]. These works meta-train small policies on tens of tasks of one robot. A complementary line improves data efficiency through pre-trained visual representations [31, 32] or by fine-tuning generalist policies [3, 22, 37, 39, 41], typically with parameter-efficient adapters [17, 23]. Their fine-tuning sets still contain tens to hundreds of demonstrations per task; the regime of three demonstrations per task with a billion-parameter model has, to our knowledge, not been studied. Our method is orthogonal to the choice of adapter and recipe: it changes the point from which post-training starts, not how post-training proceeds.

2.2. World Action Models

World models learn to predict the consequences of actions [13, 14]. With video generation this idea has scaled to pixels. UniPi [9] plans by generating a video and recovers actions with an inverse dynamics model; GR-1 and GR-2 [8, 46] pre-train a video generator and then predict actions alongside future frames; VPP [18] feeds the rep-

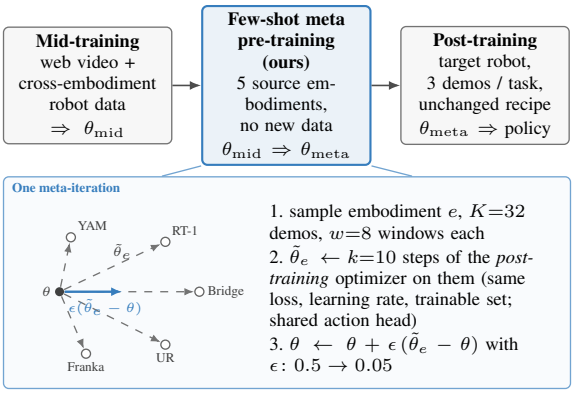


Figure 2. **Overview.** We insert a few-shot meta pre-training stage between mid-training and post-training. Each meta-iteration simulates few-shot post-training on one source embodiment (K demonstrations, k steps of the unchanged post-training recipe; dashed arrows to the adapted weights $\tilde{\theta}_e$) and moves the weights a fraction ϵ toward the adapted solution (blue arrow). The result θ_{meta} is loaded by the post-training recipe exactly like the mid-trained checkpoint.

resentations of a video diffusion model to a policy; Genie [5] and LAPA [48] learn latent actions from unlabeled videos; DreamGen [20] uses a post-trained video world model to synthesize training trajectories; and V-JEPA 2 [2] plans in a learned latent space. A growing family couples video and action generation in one network, which we refer to as world action models: UWM [53] couples video and action diffusion, WorldVLA [7] autoregresses over both modalities, Genie Envisioner [27] builds a policy on a video world model, and Cosmos [34, 36] unifies language, video, and action generation in a mixture-of-transformers backbone [26, 51]. Vision-language-action models [3, 4, 22, 35, 37, 54] are the action-only relatives of these models, trained on cross-embodiment collections [21, 38]. All of these models are post-trained on target data with tens to hundreds of demonstrations per task; we study how to make this post-training work with three. We use Cosmos3-Edge [36] as our WAM, but the meta pre-training stage only assumes a post-training recipe and a set of source robot datasets, and is agnostic to the backbone.

3. Method

We first describe the world action model and its post-training recipe (Sec. 3.1), then define the few-shot post-training problem (Sec. 3.2), and finally present the few-shot meta pre-training stage (Sec. 3.3) and how post-training uses its output (Sec. 3.4). Fig. 2 gives an overview.

3.1. Preliminaries: World Action Model and Post-Training

Model. Cosmos3-Edge [36] is a mixture-of-transformers model [26] with two 28-layer towers (hidden size 2048) coupled by joint attention: a *reasoner* tower that processes the text instruction and the conditioning tokens autoregressively, and a *generator* tower that denoises video latents and action tokens with a rectified-flow objective [28, 30]. Video is tokenized by a causal spatio-temporal VAE [45]. Actions are mapped to a unified representation (relative end-effector translation, a 6-D rotation [52], and a gripper state per arm), zero-padded to a fixed width, and projected into and out of the token stream by embodiment-specific *action heads*: one row per embodiment of an input projection, an output projection, and a modality embedding. In world-action mode the model conditions on the current observation and the instruction and jointly generates the next 16 video frames and the corresponding 16-step action chunk.

Objective. For a dataset $\mathcal{D}$ of windows (an observation, an instruction, 16 future frames, and 16 actions), the training loss is

$$\mathcal{L}(\theta;\mathcal{D}) = \lambda_v\,\mathcal{L}_{\text{video}}(\theta;\mathcal{D}) + \lambda_a\,\mathcal{L}_{\text{action}}(\theta;\mathcal{D}), \quad (1)$$

two flow-matching losses on the video latents and the action tokens, with $\lambda_v = \lambda_a = 10$ in the recipe we inherit.

Post-training recipe. Given a target dataset $\mathcal{D}_{\mathcal{T}}$, post-training minimizes Eq. (1) over a trainable subset θ of the weights: the generator tower, its timestep embedding and latent projections, and the action head of the target embodiment, 1.42B parameters in total, while the reasoner tower and the tokenizers stay frozen. The target embodiment is new, so its action-head row is initialized randomly and trained with a 5× learning-rate multiplier [36]. We write $\text{PT}_T(\theta_0;\mathcal{D}_{\mathcal{T}})$ for the weights after T steps of this recipe (AdamW [24], learning rate 5×10^{-5} with warm-up and linear decay, batches of 256 windows, T=2000) from the initialization θ_0. The standard pipeline uses the mid-trained weights, $\theta_0 = \theta_{\text{mid}}$.

3.2. Few-Shot Post-Training

We consider the regime in which $\mathcal{D}_{\mathcal{T}}$ holds only n demonstrations per task; we use n=3, *i.e.*, 30 demonstrations and about 7.6k training windows per LIBERO suite for 1.42B trainable parameters. We deliberately keep the recipe fixed, so that the only free variable is the initialization, and ask: which θ_0 maximizes the closed-loop performance of $\text{PT}_T(\theta_0;\mathcal{D}_{\mathcal{T}})$ when $\mathcal{D}_{\mathcal{T}}$ is small? Fig. 1 shows that θ_{mid} is a poor answer. Mid-training minimizes $\mathbb{E}_e[\mathcal{L}(\theta;\mathcal{D}_e)]$ over a mixture of embodiments e, *i.e.*, it looks for one set of weights that is good for all of them at once. Few-shot post-training needs something else: weights from which a short optimization on a *new* embodiment lands in a good region. The two coincide only if the per-embodiment solutions lie close to the joint solution. We hypothesize that they do not, and that this mismatch, together with the randomly initialized action head, is what makes post-training from θ_{mid} data hungry.

Algorithm 1 Few-shot meta pre-training with Reptile

Require: mid-trained weights θ_{mid}; source datasets $\{\mathcal{D}_e\}_{e=1}^{E}$; post-training step PT; K, w, k, N; schedule $\epsilon_1 \geq \cdots \geq \epsilon_N$

$\theta \leftarrow \theta_{\text{mid}}$; initialize the shared action head randomly
for $i = 1, \ldots, N$ **do**
  sample $e \sim \text{Unif}\{1, \ldots, E\}$ and K demonstrations of $\mathcal{D}_e$
  $\mathcal{S}_e \leftarrow w$ windows from each demonstration
  $\tilde{\theta}_e \leftarrow \text{PT}_k(\theta; \mathcal{S}_e)$ ▷ k steps of the post-training recipe, fresh optimizer state
  $\theta \leftarrow \theta + \epsilon_i (\tilde{\theta}_e - \theta)$
end for
return $\theta_{\text{meta}} = \theta$ ▷ trainable set and shared action head

3.3. Few-Shot Meta Pre-Training

We therefore add a stage that directly optimizes θ for being a good starting point of PT on an unseen embodiment, using source embodiments as stand-ins for the target. The procedure is summarized in Alg. 1.

Source episodes. We use five robot datasets that are part of the mid-training mixture, each recorded with a different embodiment: RT-1 (Google Robot) [4], BridgeData V2 (WidowX) [44], the UR-5e and the Franka subsets of RoboMIND [47], and the bimanual YAM data of MolmoAct [25], whose 20-D actions exercise the padded action representation. Every source is read through the same windowing and transformation pipeline as the target, so meta-training samples have exactly the format that post-training samples have. An episode samples an embodiment $e \sim \text{Unif}\{1, \ldots, 5\}$ and K demonstrations of e, and takes w windows from each of them, giving a support set $\mathcal{S}_e$ of Kw windows. Sampling whole demonstrations rather than windows matters: the downstream data consists of a few complete trajectories, and the support set should have the same structure, a few trajectories with many windows each.

Shared action head. The target embodiment arrives without an action head. To meta-learn the head as well, all source embodiments share one spare action-head row during meta pre-training, initialized from scratch like the downstream head. At post-training time this row is copied into the row of the target embodiment, so the target head starts from a meta-learned initialization instead of a random one; accordingly, it is trained with the base learning rate instead of the $5\times$ multiplier.

Inner loop: the post-training recipe. From the current weights θ, the inner loop runs k steps of exactly the post-training optimizer on $\mathcal{S}_e$: the same trainable set, the loss of Eq. (1) with the same λ_v, λ_a, AdamW with the same learning rate and weight decay, the same gradient clipping, and a short warm-up; the optimizer state is reset at every episode. We write $\tilde{\theta}_e = \text{PT}_k(\theta; \mathcal{S}_e)$. Each inner step covers the whole support set, so k steps correspond to k passes over the K demonstrations, a miniature of the downstream regime, in which 2000 steps cover the 30 demonstrations about 70 times.

Outer update. Reptile [33] then interpolates between the current and the adapted weights,

$$\theta \leftarrow \theta + \epsilon\,(\tilde{\theta}_e - \theta), \tag{2}$$

with ϵ annealed linearly from 0.5 to 0.05 over the N episodes. A fixed point of Eq. (2) is a point whose expected displacement under k-step adaptation vanishes, *i.e.*, a centroid of the per-embodiment k-step solutions, from which the adaptation to each embodiment is short. Nichol *et al.* [33] further show that for $k > 1$ the expected update contains, besides the average gradient, a term that increases the inner product between the gradients of different mini-batches of the same task. In our setting that term rewards weights for which fitting a few windows of an embodiment also reduces the loss on its remaining windows, which is precisely the generalization that few-shot post-training requires.

Why Reptile. Three properties make Reptile the right tool at this scale. *(i) Long inner horizon.* The downstream adaptation is 2000 optimizer steps. First-order MAML, whose meta-gradient is the query-set gradient at the adapted point, is optimized for the short inner horizon it is trained with; in preliminary experiments with a 5-step inner loop it produced initializations whose offset from θ_{mid} the 2000-step post-training neither used nor undid. Reptile has no query set, and its k is limited only by compute. *(ii) Memory.* Eq. (2) needs one extra copy of the trainable parameters and nothing else; the model's own FSDP-sharded fp32 parameters [50] are the fast weights, and the standard trainer, optimizer, activation checkpointing, and checkpoint code run the inner loop unchanged. This is what makes meta-training the entire 1.42B-parameter trainable set affordable on four 48 GB GPUs, whereas an inner loop with per-step fast weights or a query gradient at the adapted point would not fit. *(iii) Compatibility.* The outer update is a convex combination of weights, so θ remains a valid checkpoint of the unchanged architecture at every iteration and is loaded by the post-training recipe without modification. Meta-training the whole trainable set rather than only the heads is important: in preliminary experiments, meta-learning only the action heads (an ANIL-style variant [40]) gave at most marginal gains (62.6% versus 58.0% on LIBERO-10, within evaluation noise), because full-parameter post-training then overwrites the small meta-learned subset.

3.4. Post-Training from the Meta Pre-Trained Weights

Post-training from θ_{meta} differs from the baseline in exactly two places: the trainable set is loaded from θ_{meta} instead of θ_{mid}, and the target action-head row is initialized from the shared meta-learned head and trained with a $1\times$ instead of a $5\times$ learning-rate multiplier. The frozen reasoner tower and tokenizers, the data pipeline, the loss, the optimizer, the schedule, the number of steps, and the inference procedure are unchanged. The same θ_{meta} is used for every target suite and every data budget in Sec. 4; meta pre-training is a one-time cost of about 120 GPU-hours that is amortized over all downstream targets.

4. Experiments

4.1. Setup

Benchmark. We evaluate on the four LIBERO suites [29], each with 10 language-conditioned manipulation tasks: LIBERO-Spatial (same objects, different layouts), LIBERO-Object (different objects, same layout), LIBERO-Goal (same scene and objects, different goals), and LIBERO-10 (long-horizon, two-stage tasks). We use the 20 Hz LeRobot release of the demonstrations [6], which contains 379, 428, 432, and 454 successful demonstrations for LIBERO-10, -Goal, -Spatial, and -Object.

Few-shot protocol. For every suite we select three demonstrations per task with a fixed random seed, 30 demonstrations in total, and keep this selection fixed for all methods. Five further demonstrations per task, disjoint from the training set, are held out to monitor the validation loss; they are never used for training or for checkpoint selection, and we always evaluate the final checkpoint of each run.

Models. Cosmos3-Edge-PT post-trains the mid-trained Cosmos3-Edge checkpoint with the standard recipe; this is the baseline of Fig. 1. Cosmos3-Edge-META-PT post-trains from our meta pre-trained checkpoint θ_{meta} as described in Sec. 3.4. The two runs of a suite share every setting, including the training subset and the seed, and differ only in the initialization and in the learning-rate multiplier of the action head ($5\times$ for the fresh head, $1\times$ for the meta-learned head).

Post-training details. The policy observes the third-person and the wrist camera at 256×256, concatenated side by side, and the instruction, and generates the next 16 frames together with a 16-step chunk of 10-D actions (relative end-effector translation, 6-D rotation, and gripper), normalized with the quantile statistics of the suite. We use AdamW with learning rate 5×10^{-5}, weight decay 0.05, $\beta = (0.9, 0.99)$, 200 warm-up steps and linear decay to zero at 2000 steps, batches of 256 windows, an exponential moving average of the weights, bf16 compute with fp32 master weights, and FSDP over two 48 GB GPUs; the instruction is dropped for 10% of the samples.

Model	LIBERO-10	Goal	Spatial	Object	Avg.
PT, all demonstrations	87.2	97.4	97.8	98.8	95.3
PT (baseline)	58.0	48.8	72.0	82.8	65.4
META-PT (ours)	**69.4**	**60.8**	**72.6**	**85.2**	**72.0**
Δ	+11.4	+12.0	+0.6	+2.4	+6.6

Table 1. **Few-shot post-training on LIBERO (3 demonstrations per task).** Success rate (%) over 50 trials on each of the 10 tasks of a suite. PT is Cosmos3-Edge-PT, post-trained from the mid-trained weights; META-PT is Cosmos3-Edge-META-PT, post-trained from our meta pre-trained weights. Both are post-trained with the identical recipe on the identical 30 demonstrations and differ only in the initialization. The gray row is the baseline recipe trained on all demonstrations of the suite, for reference.

Meta pre-training details. We run $N{=}1000$ meta-iterations over the five source embodiments with $K{=}32$ demonstrations and $w{=}8$ windows per demonstration (256 windows per episode), $k{=}10$ inner AdamW steps at the post-training learning rate 5×10^{-5} and weight decay 0.05, two warm-up steps, gradient clipping at 1.0, and a fresh optimizer state per episode; ϵ decays linearly from 0.5 to 0.05. The run takes 1.5–2 minutes per meta-iteration on four 48 GB GPUs, 25–30 hours in total, with a peak memory below 30 GB per GPU, and consumes about 32k demonstration samples, a negligible fraction of the mid-training data. The final θ_{meta} is used for all experiments below.

Evaluation. We run the policy closed loop in the LIBERO simulator, 50 trials per task with the benchmark's fixed initial states, *i.e.*, 500 episodes per suite and per model. Each predicted 16-step action chunk is executed before the policy is queried again; actions are sampled with 30 denoising steps and without classifier-free guidance. Episodes time out after 520, 300, 220, and 280 control steps for LIBERO-10, -Goal, -Spatial, and -Object. We report the success rate in percent. With 500 Bernoulli trials the standard error at a success rate of 60% is about 2.2 points, so differences of fewer than about four points should be read as ties.

4.2. Few-Shot Post-Training

Tab. 1 reports the main result. Starting post-training from θ_{meta} instead of θ_{mid} improves all four suites, by 6.6 points on average. The gains are largest where the baseline is weakest: +11.4 points on LIBERO-10 and +12.0 points on LIBERO-Goal, roughly five standard errors each, while LIBERO-Spatial and LIBERO-Object, on which the baseline already reaches 72% and 83% from 30 demonstrations, improve by 0.6 and 2.4 points, within the noise of the evaluation. Averaged over the suites, meta pre-training closes 22% of the gap between few-shot post-training (65.4%) and post-training on all demonstrations (95.3%), without touch-

#	Task (abbreviated instruction)	PT	META-PT	Δ
1	alphabet soup & tomato sauce in basket	20	**42**	+22
2	cream cheese & butter in basket	76	**80**	+4
3	turn on stove, moka pot on it	50	**60**	+10
4	black bowl in bottom drawer, close it	56	**66**	+10
5	white mug on left plate, other on right	50	**62**	+12
6	book in back compartment of caddy	74	**78**	+4
7	white mug on plate, pudding to its right	54	**74**	+20
8	alphabet soup & cream cheese in basket	58	**70**	+12
9	both moka pots on the stove	60	**78**	+18
10	yellow-white mug in microwave, close it	82	**84**	+2
	All tasks	58.0	**69.4**	+11.4

Table 2. **Per-task success rate (%) on LIBERO-10 with 3 demonstrations per task**, 50 trials per task. PT and META-PT denote Cosmos3-Edge-PT and Cosmos3-Edge-META-PT. The meta pre-trained initialization improves every task.

Training demonstrations	PT	META-PT	Δ
3 per task (30)	58.0	**69.4**	+11.4
all (379)	87.2	**94.6**	+7.4

Table 3. **Beyond the few-shot regime: LIBERO-10 with all demonstrations.** Success rate (%) over 500 episodes; PT and META-PT as in Tab. 1. Post-training from the meta pre-trained weights also improves the full-data setting, removing more than half of the remaining failures.

ing the post-training recipe, the model, or the data; the remaining gap shows that few-shot post-training of WAMs is far from solved.

Per-task behavior. Tab. 2 breaks LIBERO-10 down by task. The improvement is not concentrated in a few tasks: all ten tasks improve, by 2 to 22 points. The largest gains appear on tasks with two sequential pick-and-place segments that involve visually similar containers (tasks 1, 7, and 9), where the baseline tends to complete only the first segment within the time limit, while the tasks on which the baseline is already strong (tasks 2, 6, and 10) change little.

4.3. Beyond the Few-Shot Regime

Does the meta pre-trained initialization only act as a regularizer for tiny datasets, or is it a better starting point in general? Tab. 3 repeats the LIBERO-10 experiment with all 379 demonstrations and the same 2000-step schedule. Post-training from θ_{meta} raises the success rate from 87.2% to 94.6%, *i.e.*, the failure rate drops from 12.8% to 5.4%. The gain is smaller than in the few-shot regime, as expected when the target data can itself move the weights far, but it is well outside the evaluation noise. We conclude that θ_{meta} is a better initialization for post-training in general, and that its advantage grows as the target data shrinks.

4.4. Discussion and Limitations

What the gains do and do not show. The two models of Tab. 1 share the training subset, the recipe, the seed, and the evaluation episodes, so the comparison is paired, but each number is a single training run evaluated on 500 episodes; we therefore do not interpret the sub-four-point differences on LIBERO-Spatial and LIBERO-Object. The results also do not yet separate the contribution of the Reptile rule from that of additional exposure to the five source datasets, since θ_{meta} has seen about 32k more demonstration samples than θ_{mid}; a control that consumes the same episodes with plain joint training (k=1, ϵ=1) is the natural next experiment.

Where the remaining failures are. Inspecting the rollouts of the few-shot policies on LIBERO-Goal and LIBERO-Spatial, whose ten tasks share one scene, we find that most failures are not imprecise motion but the execution of a different task of the same scene, or the manipulation of the wrong instance of a repeated object. Three demonstrations per task appear to be enough to adapt the dynamics and the embodiment, which is what our source episodes exercise, but not to ground ten instructions in one scene. Meta-training episodes that contain several instructions of the same scene are a promising direction.

Scope. We study one backbone and one model size, one meta pre-training budget, and simulated evaluation; the stage itself only assumes a post-training recipe and a set of source robot datasets and should transfer to other WAMs and VLAs.

5. Conclusion

We showed that the standard post-training recipe of a world action model breaks down when only three demonstrations per task are available, losing 16 to 49 points on the four LIBERO suites, and traced the problem to a mismatch between the objective of mid-training, which seeks weights that are good for a mixture of embodiments, and the need of few-shot post-training, which requires weights from which a short fine-tune on a new embodiment succeeds. Few-shot meta pre-training closes part of this gap with a Reptile stage that repeatedly simulates few-shot post-training on source embodiments already present in mid-training and moves the entire trainable set of the recipe toward the adapted solutions. The stage needs no new data, runs on the unchanged sharded training stack, and produces a checkpoint that the unchanged post-training recipe consumes. It improves all four suites in the few-shot regime, every task of LIBERO-10, and the full-data setting as well. Separating the meta-learning rule from the extra data exposure, meta-training with multi-instruction episodes to address instruction grounding, and scaling the stage to larger backbones are the next steps.

References

[1] Antreas Antoniou, Harrison Edwards, and Amos Storkey. How to train your MAML. In *International Conference on Learning Representations (ICLR)*, 2019. 2

[2] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 3

[3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 2, 3

[4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023. 3, 4

[5] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024. 3

[6] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. https://github.com/huggingface/lerobot, 2024. 5

[7] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21734*, 2025. 1, 3

[8] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, Hanbo Zhang, and Minzhao Zhu. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024. 2

[9] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 1, 2

[10] Yan Duan, Marcin Andrychowicz, Bradly Stadie, Jonathan Ho, Jonas Schneider, Ilya Sutskever, Pieter Abbeel, and Wojciech Zaremba. One-shot imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2

[11] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*, 2017. 2

[12] Chelsea Finn, Tianhe Yu, Tianhao Zhang, Pieter Abbeel, and Sergey Levine. One-shot visual imitation learning via meta-learning. In *Conference on Robot Learning (CoRL)*, 2017. 2

[13] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. 2

[14] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020. 2

[15] Kourosh Hakhamaneshi, Ruihan Zhao, Pieter Abbeel, Sergey Levine, and Ofir Nachum. Hierarchical few-shot imitation with skill transition models. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[16] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 44(9):5149–5169, 2022. 2

[17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[18] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning (ICML)*, 2025. 2

[19] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn. BC-Z: Zero-shot task generalization with robotic imitation learning. In *Conference on Robot Learning (CoRL)*, 2021. 2

[20] Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, et al. DreamGen: Unlocking generalization in robot learning through neural trajectories. *arXiv preprint arXiv:2505.12705*, 2025. 3

[21] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024. 3

[22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024. 2, 3

[23] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 2

[24] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. 3

[25] Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Wang, Sangho Lee, et al. MolmoAct: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025. 4

[26] Weixin Liang, Lili Yu, Liang Luo, Srinivasan Iyer, Ning Dong, Chunting Zhou, Gargi Ghosh, Mike Lewis, Wen-tau Yih, Luke Zettlemoyer, and Xi Victoria Lin. Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models. *arXiv preprint arXiv:2411.04996*, 2024. 3

[27] Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, Liliang Chen, Shuicheng Li, Yang Wang, Shuangjie Chen, et al. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025. 3

[28] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[29] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. 1, 5

[30] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[31] Arjun Majumdar, Karmesh Yadav, Sergio Arnaud, Yecheng Jason Ma, Claire Chen, Sneha Silwal, Aryan Jain, Vincent-Pierre Berges, Pieter Abbeel, Jitendra Malik, Dhruv Batra, Yixin Lin, Oleksandr Maksymets, Aravind Rajeswaran, and Franziska Meier. Where are we in the search for an artificial visual cortex for embodied intelligence? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2

[32] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2022. 2

[33] Alex Nichol, Joshua Achiam, and John Schulman. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*, 2018. 2, 4

[34] NVIDIA. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025. 3

[35] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. 3

[36] NVIDIA. Cosmos 3: Omnimodal world models for physical AI. Technical report, NVIDIA, 2026. Technical report, https://research.nvidia.com/labs/cosmos-lab/cosmos3/. 1, 3

[37] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024. 2, 3

[38] Open X-Embodiment Collaboration. Open X-embodiment: Robotic learning datasets and RT-X models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. 3

[39] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 2

[40] Aniruddh Raghu, Maithra Raghu, Samy Bengio, and Oriol Vinyals. Rapid learning or feature reuse? Towards understanding the effectiveness of MAML. In *International Conference on Learning Representations (ICLR)*, 2020. 2, 4

[41] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[42] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2

[43] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Koray Kavukcuoglu, and Daan Wierstra. Matching networks for one shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. 2

[44] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Conference on Robot Learning (CoRL)*, 2023. 4

[45] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. 3

[46] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2024. 1, 2

[47] Kun Wu, Chengkai Zhu, Jiaming Hou, Zhengping Li, Ningbo Liu, Zhongyang Zhang, Yanjiang Cheng, et al. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024. 4

[48] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejune Joo, Jianwei Lin, Jiyoung Ahn, Linxi Fan, Yuke Kim, et al. Latent action pretraining from videos. In *International Conference on Learning Representations (ICLR)*, 2025. 3

[49] Tianhe Yu, Chelsea Finn, Annie Xie, Sudeep Dasari, Tianhao Zhang, Pieter Abbeel, and Sergey Levine. One-shot imitation from observing humans via domain-adaptive meta-learning. In *Robotics: Science and Systems (RSS)*, 2018. 2

[50] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023. 2, 4

[51] Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe

Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. *arXiv preprint arXiv:2408.11039*, 2024. 3

[52] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 3

[53] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Robotics: Science and Systems (RSS)*, 2025. 1, 3

[54] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. 3